**SDC HACKATHON 2026**

# Phase 2: Final Project & Prototype Report

Hosted by Xiamen University Malaysia (XMUM) AI Club

Submission Deadline: 1 November 2026

Instructions: Complete every field below and submit as a single PDF document named "SDC2026_TeamName_Final.pdf" together with your pitch deck and a working deployment link. Answers must comply with the word limits stated for each open-ended question. Section 3 requires concrete, verifiable evidence. Unsupported claims will not be credited by judges.

## 1. Team & Project Overview

| | |
|---|---|
| **Team Name** | VBall |
| **Project / Product Name** | SMErgers |
| **Final Live Deployment URL** | https://sm-ergers-1.replit.app |
| **Codebase Repository URL** | https://github.com/Anonys-sky/SMErgers |
| **Name (Leader)** | Vincent Chew Wen Jie |
| **Email & phone (Leader)** | vinchew00@gmail.com \| +6011-20208512 |

## 2. Project Description

***[Maximum 250 words]***

SMErgers helps small event companies turn changes agreed in a job's WhatsApp conversation into traceable quote revisions and a draft final bill. After a signed quote, clients often request extras in chat. In a synthetic dinner example, RM3,860 of changes, 11.2% of a RM34,530 quote, had no formal record. This example illustrates the workflow; it is not a measured industry loss rate.

The application imports a WhatsApp text export or reads messages through an optional group connector. Its rules reader, or a selected Llama 3.1 8B reader hosted on RunPod, identifies candidate changes and points to the supporting messages. Code checks prices against the conversation and quote. A project manager reviews each item, a director handles large or unusual decisions, and the client confirms or disputes it. Confirmed items generate a new quote version, draft bill, CSV and activity log. The bill remains a draft; the prototype does not issue invoices or move money.

The current prototype uses synthetic data, role workspaces, a guided interface and server enforced approval rules. It is built with Python, Flask and Jinja, developed and hosted through Replit. Its aim is to reduce missed billable work and administrative effort for SMEs, supporting SDG 8.2 and 8.3. The next step is a consent based pilot with real event teams and timed measurement.

## 3. Upgraded Product & Technical Execution

### 3.1 Final System Architecture

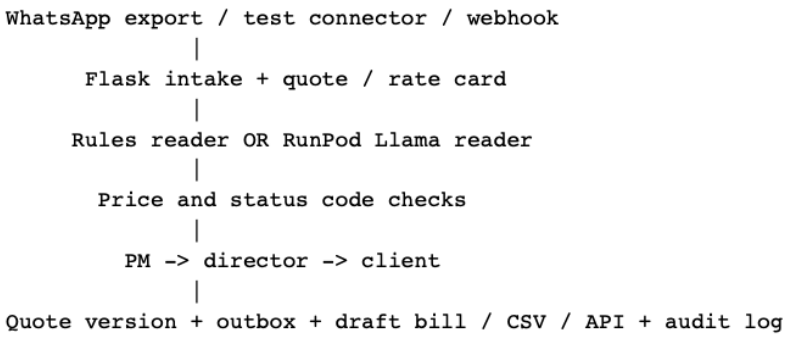
*Provide a detailed description (and, if possible, an attached technical diagram) of your fully built system. Explain how your application integrates GenAI APIs, database structures, and backend/frontend frameworks into one unified solution.*

Input: WhatsApp text export, signed quote and rate card. The optional WhatsApp Web test connector can select a group and poll new messages; the app also implements Meta Cloud API webhook verification and reception. Outgoing group summaries are recorded in a prototype outbox and are not delivered through the official API. The production WhatsApp connection requires a verified business account and further integration.

Processing: Flask parses messages and passes them to a default rules reader or, when selected, an AI reader calling a RunPod Serverless vLLM endpoint running pre-trained Llama 3.1 8B Instruct. The model proposes changes and evidence references only. Deterministic checks validate prices and states; rules are the fallback if the endpoint is unavailable.

Workflow: project manager review, conditional director approval under each business's sign-off limit, and client confirmation or dispute are checked on the server. Confirmed changes create quote versions and feed the draft bill, CSV, read-only job API and SHA-256 chained activity log. Flask and Jinja render the browser

interface. Synthetic JSON files seed an in-memory store, which resets on restart; there is no persistent production database yet.




### 3.2 Implementation & Hackathon Deliverables

*Describe what was successfully built, integrated, and deployed during the 48-hour physical sprint. Highlight any specific problem-solving steps your team took to pivot or improve the project from the Phase 1 preliminary design.*

***[Maximum 300 words]***

During the 48-hour sprint, we improved SMErgers' working prototype so an event team can take a change from a WhatsApp conversation through to a draft bill. The app shows the messages behind each proposed change. A project manager checks it, a director approves changes that need extra sign-off, and the client can confirm or dispute it. Confirmed changes then appear in a revised quote and draft bill.

We also made the workflow easier to use. Businesses can set their own director approval limit, directors have a clearer list of decisions to make, and new users get a guided introduction. We added an optional connection that reads new messages from a selected WhatsApp group, replacing the simulated "live group" used in the earlier demo.

To check how well the system works, we tested it on a new set of synthetic conversations and replayed the full approval process to count what the software found, what people had to correct, and what reached the draft bill. These tests exposed mistakes the reader can make, so we kept human checks and client confirmation before billing.

The prototype runs on Replit.

## 4. Environmental & SDG Impact Verification

*Validate the real-world impact of your built solution. Show concrete data that validate your environmental or social value proposition, compared against the baseline and KPI defined in previous phase submission.*

*The data and evidence you provide in this section will be used to assess* ***SDG Impact, Technical Execution and Innovation scores.*** *Vague claims or missing verification will weaken your score across all three categories.*

| **Metric Category** | **Your Response** |
|---|---|
| **Target SDG Metric Achievement**<br>*Direct comparison between your Phase 1 baseline and your Phase 2 finalized prototype numbers*<br>*e.g., "Achieved a 24.5% reduction in computational execution overhead against the 20% initial target."* | SDG 8.2 and 8.3. Phase 1 targets: at least 95% of agreed billable changes on the final bill, and 50% less final bill preparation time. In a synthetic four chat replay, a careful simulated team brought 15/15 agreed billable changes (100%) to the draft bill. This meets the coverage target in simulation only; no real company outcome is verified.<br><br>Phase 1 assumed that roughly 75% of chat agreed changes reach an event company's final bill, based on one team member's experience, explicitly unverified. Phase 2 replay: 15/15 in a draft bill after human corrections, versus 5/15 (33%) that the rules reader got right on its own. The 75% estimate and 100% replay are different populations and cannot establish a real world improvement. |
| **Verification Evidence Method**<br>*How was this data verified?*<br>*e.g., codebase algorithmic profiling logs, hardware power-meter simulation readings, or database transaction counters.* | Run python src/benefit.py --split blind2. Four synthetic chats and answer keys drive the actual workflow, quote and bill code; the script reads the resulting bill and activity log. It counts 9 reader errors corrected by people, 1 missed change added, 1 item rejected and 49 human actions. The clean second blind evaluation separately scores the AI reader at 27/36 fully right (75%) on synthetic chats, not real use. |
| **Socio-Economic / Civic Value** | A small firm can preserve evidence for changes, reduce forgotten extras and give clients a chance to dispute |

| Metric Category | Your Response |
|---|---|
| *How does your deployed application translate into a scalable, real-world solution capable of being supported by civic or corporate partners?* | charges. A real pilot must measure billed change coverage, time and dispute rates before any general impact claim. |
| **Deployment & Commercial Feasibility**<br>*e.g., modeled as an API service plugin designed for easy integration into existing corporate or institutional systems.* | A Flask job API and CSV support future integration with accounting or SME systems. Replit hosts this project and RunPod supplies optional AI inference. Commercial use needs persistent storage, authentication, official messaging integration, consent, retention rules and a pilot. |

### 4.1 Mathematical Proof / Code Logic Snippet (Optional, but holds bonus marks)

*Paste the specific code section, mathematical formula, simulation log output, or calculation logic that proves the impact claims made above.*

*This proof is graded across three categories at once:* ***Impact and Feasibility*** *(does the math support your claim),* ***Technical Execution*** *(does it reflect what you actually built), as well as* ***Innovation and Creativity*** *(is the approach original).] Do not leave this section generic or copied from Phase 1.*

We tested the built workflow on four synthetic conversations with 15 agreed, billable changes. The replay sends detected changes through project manager review, director approval where required, and client confirmation. It also has a simulated careful reviewer correct nine changes and add one that the rules reader missed.

The billing code includes only confirmed changes:

```
7  BILLABLE = {"confirmed"}
```

```
42      for key, item in job.items.items():
43          if item.state not in BILLABLE:
44              continue
```

(code located in src/bill.py)

The replay produced:

| | |
|---|---|
| Agreed, billable changes: | 15 |
| Right by rules reader alone: | 5 |
| Corrected by simulated reviewer: | 9 |
| Added by simulated reviewer: | 1 |
| Confirmed changes on draft bill: | 15 |
| Draft bill changes total: | RM5,515 |
| Answer-key changes total: | RM5,515 |
| Recorded human actions: | 49 |

Prototype coverage: 15 ÷ 15 = 100%, compared with the Phase 1 target of 95%. This proves the workflow can carry corrected changes into a draft bill when a careful reviewer finds every error. It does not measure how often real staff would do so.

The same replay estimates 99 minutes to prepare the bills manually and 21 minutes with SMErgers: (99 − 21) ÷ 99 = 78.8% less time, above the Phase 1 target of 50%. These minutes come from stated reading and action-time assumptions in src/benefit.py (line 34), not a timed test with people. Reproduce the result with python src/benefit.py --split blind2.

## 5. Market Viability & Future Scalability Plan

*SDC aims to encourage solutions that transition from innovations into market-ready applications. If given access to corporate funding, industry grants, or partnership opportunities (e.g., through SDC's Rotary network or joint research initiatives at XMUM), how would your team scale this prototype to process real-world data at volume?*

***[Maximum 250 words]***

The first buyers would be owner managed event firms that already coordinate jobs in WhatsApp and create quotes manually. A pilot with 3 to 5 consenting firms would compare the same jobs processed manually and in SMErgers. We would record the share of agreed changes that reach the final bill, minutes spent preparing it, correction and dispute rates, and missed changes found by a second reviewer. We would report actual distributions and failures, not extrapolate the synthetic replay as field performance.

Funding would first support a persistent database for jobs, messages, quote versions and the activity log; real accounts and role permissions; encrypted storage and access controls; retention and deletion flows; and legal review for Malaysian personal data. We would integrate the official WhatsApp Business API where its group and account constraints allow it, while keeping consent based text import for other teams. Outgoing messages would then be delivered rather than shown only in a prototype outbox. RunPod workers could scale independently of the Flask app, with the rules reader available during AI delays or outages.

The read-only job API and CSV make an accounting or Orbyte integration feasible; we would test it with a partner. Proposed pricing: free for one job at a time, RM100 a month for 10 jobs, RM250 for unlimited jobs. The AI reader cost about US$0.02 per chat read. Real recovery rate and willingness to pay are unproven; 6 of 7 people in our own-network survey chose a paid plan.

## 6. Final Submission Checklist

*Tick all boxes that apply before submitting.*

- ☑ The application is fully compiled, live, and accessible via the deployment link provided in Section 1.
- ☑ The GitHub (or equivalent) repository contains a clean, well-documented README with installation instructions.
- ☑ The GitHub (or equivalent) repository shows the history of incremental code updates made throughout the official live hacking window.
- ☑ The final pitch deck has been exported and saved as a flattened PDF file.
- ☑ All fields in this template have been completed and word limits respected.

## 7. IP & Originality Declaration

### 7.1 Pre-Existing / Prior-Owned Work

Does this submission include any code, design, research, or other work that any team member created or owned before the start of this hackathon?

☐ Yes ☑ No ☐ No change from Phase 1 declaration

If yes, briefly describe what was pre-existing and which team member(s) owned it:

### 7.2 Third-Party Code, Datasets, or Models

Does your project use any third-party code libraries, open-source frameworks, public/private datasets, or pre-trained AI models that your team did not create?

☑ Yes ☐ No ☐ No change from Phase 1 declaration

If yes, list them below (open-source use is permitted provided it is declared and properly licensed/attributed):

| Name | Type (code/dataset/model) | License |
|---|---|---|
| Llama 3.1 8B Instruct (Meta) | model (pre-trained) | Llama 3.1 Community License |
| vLLM; Ollama | code | Apache-2.0; MIT |
| Flask, Jinja2, MarkupSafe, Werkzeug | code | BSD-3-Clause |
| whatsapp-web.js; Puppeteer | code | Apache-2.0 |
| qrcode; Node.js; pytest | code | MIT |
| tzdata; Pygments | code | Apache-2.0; BSD-2-Clause |
| DM Sans, Manrope, IBM Plex Mono (Google Fonts) | font (loaded from Google Fonts) | SIL Open Font License 1.1 |

Full versions and licences: Technical & Tools Declaration, sections 2 to 4.

### 7.3 Employment-Related IP Conflicts

Is any team member currently bound by an employment, internship, or consulting agreement under which their employer could claim ownership of work they create during this hackathon?

☐ Yes ☑ No ☐ No change from Phase 1 declaration

If yes, please describe the nature of the agreement and which member it concerns:

## 8. Declaration

By submitting this report, the team leader confirms that the deployed application and data presented above are the original work of the team, completed within the official SDC 2026 hackathon period.

[Signature]



[Date]

**31 October 2026**